

Pistachio Supply Chain Optimization

Raissa Gonçalves Diniz¹ and André Costa Batista^{1,2}

¹Operations Research and Complex Systems Laboratory, Universidade Federal de Minas Gerais, Brazil.

²Department of Electrical Engineering, Universidade Federal de Minas Gerais, Av. Antônio Carlos 6627, Belo Horizonte, 31270-010, Minas Gerais, Brazil.

Contributing authors: raissagd@ufmg.br; andre-costa@ufmg.br;

Abstract

The increasing focus on sustainability in agriculture has led to concerns about managing agricultural by-products and optimizing resource usage. This study addresses these challenges by proposing a mixed linear mathematical model for optimizing the pistachio supply chain network. The model integrates forward and reverse logistics to reduce environmental impact, including the return of by-products to the network through composting. To minimize total fixed and variable costs, we employ the Variable Neighborhood Search (VNS) algorithm and compare its performance with an exact algorithm (Gurobi) and a population-based metaheuristic (Genetic Algorithm). Our results demonstrate the efficiency and scalability of VNS, especially for larger problem instances, making it a viable solution for sustainable agricultural supply chain optimization.

Keywords: Optimization, Supply chain, Metaheuristics, Environmental impact, Variable Neighborhood Search

1 Introduction

The growing role of agriculture in the economic development of many countries highlights the importance of integrating this sector with other industries to stimulate overall economic growth. Agriculture not only supports food security but also acts as a major contributor to exports, generating significant income and bolstering other sectors. Among various agricultural products, pistachios stand out due to their high economic and nutritional value. As major producers such as the US, Turkey, and Iran

lead global production, the strategic importance of pistachios in international trade continues to rise.

With the increasing global demand for pistachios, the need for efficient supply chain networks becomes more critical. Proper supply chain management is vital for maintaining product quality, minimizing environmental impacts, and ensuring the smooth flow of goods from production to consumers. In particular, pistachio production faces challenges related to waste management, as large volumes of by-products such as hulls and shells are generated during processing. Addressing these environmental concerns, along with optimizing logistics, becomes crucial for sustaining both the economic and ecological viability of the pistachio industry.

Given the complex interplay of factors influencing pistachio production and trade, this study focuses on designing an optimal supply chain network for pistachios. Specifically, we explore the development of a closed-loop supply chain (CLSC), which integrates forward and reverse logistics to minimize waste and maximize resource utilization. While there has been extensive research on CLSCs for other agricultural products, pistachios remain underexplored. Therefore, this work aims to fill that gap by proposing a quantitative model for the design of a pistachio CLSC network. Through the application of several meta-heuristic and hybrid algorithms, this study seeks to provide an effective solution for enhancing the sustainability and efficiency of the pistachio supply chain.

2 Problem definition

The proposed network includes producers, processing centers, pistachio factories, oil extraction centers, composting centers, and customers. Pistachios are transported from orchards to processing centers, producing open-mouth pistachios, raw kernels, and waste, which are then sent to factories, oil extraction centers, and composting centers, respectively. Open-mouth pistachios are roasted, packed, and shipped to customers, while pistachio oil is delivered to oil customers and cosmetic factories. Waste is converted into compost for distribution. The model aims to minimize total costs (1), including opening, transportation, and production costs, under the following capacity and demand constraints with no allowed shortages:

$$\min z_1 + z_2 + z_3 \quad (1)$$

$$\text{s.t.} \quad \sum_{j=1}^J X_{ij} \leq Cpa_i, \quad \forall i \in I \quad (2)$$

$$\sum_{i=1}^I X_{ij} \leq Cpu_j \times U_j, \quad \forall j \in J \quad (3)$$

$$\sum_{e=1}^E Ow_{eq} + \sum_{j=1}^J Gw_{jq} \leq Cpy_q \times T_q, \quad \forall q \in Q \quad (4)$$

$$\sum_{j=1}^J Go_{jk} \leq Cpw_k \times W_k, \quad \forall k \in K \quad (5)$$

$$\sum_{j=1}^J Gr_{je} \leq Cpr_e \times R_e, \quad \forall e \in E \quad (6)$$

$$\sum_{e=1}^E Oen_2 \leq Cpv_s \times V_s, \quad \forall s \in S \quad (7)$$

$$\sum_{k=1}^K Go_{jk} + \sum_{e=1}^E Gr_{je} + \sum_{q=1}^Q Gw_{jq} \leq \sum_{i=1}^I X_{ij}, \quad \forall j \in J \quad (8)$$

$$\sum_{k=1}^K Go_{jk} = (1 - \beta) \sum_{i=1}^I X_{ij} \theta_1, \quad \forall j \in J \quad (9)$$

$$\sum_{e=1}^E Gr_{je} = (1 - \beta) \sum_{i=1}^I X_{ij} \theta_2, \quad \forall j \in J \quad (10)$$

$$\sum_{q=1}^Q Gw_{jq} = \sum_{i=1}^I X_{ij} \theta_3, \quad \forall j \in J \quad (11)$$

$$\theta_1 + \theta_2 + \theta_3 = 1 \quad (12)$$

$$\sum_{n_1=1}^{N_1} P_{kn_1} \leq \sum_{j=1}^J Go_{jk} \times \gamma k, \quad \forall k \in K \quad (13)$$

$$\sum_{n_2=1}^{N_2} Oen_2 + \sum_{s=1}^S Oces \leq \sum_{j=1}^E Gr_{je} \times (1 - \lambda), \quad \forall e \in E \quad (14)$$

$$\sum_{q=1}^Q Ow_{eq} \leq \sum_{j=1}^J Gr_{je} \times \lambda k, \quad \forall e \in E \quad (15)$$

$$\sum_{n_3=1}^{N_3} L_{sn_1} \leq \sum_{e=1}^E Oces \times \gamma s, \quad \forall s \in S \quad (16)$$

$$\sum_{m=1}^M D_{qm} \leq \left(\sum_{j=1}^J Gw_{jq} + \sum_{e=1}^E Ow_{eq} \right) \times \gamma q, \quad \forall q \in Q \quad (17)$$

$$\sum_{k=1}^K P_{kn_1} \leq Dp_{n_1}, \quad \forall n_1 \in N_1 \quad (18)$$

$$\sum_{e=1}^E Oen_2 \leq Du_{n_2}, \quad \forall n_2 \in N_2 \quad (19)$$

$$\sum_{s=1}^S L_{sn_3} \leq Ds_{n_3}, \quad \forall n_3 \in N_3 \quad (20)$$

$$\sum_{q=1}^Q D_{qm} \leq Dc_m, \quad \forall m \in M \quad (21)$$

- The objective function (1) minimizes the total costs of the supply chain, including opening, transportation, and production costs. The costs are divided into three components: z_1 (22), z_2 (23), and z_3 (24), which represent the costs of opening facilities, transportation, and production, respectively as shown in the following equations:

$$z_1 = \sum_{j=1}^J Fu_j \times U_j + \sum_{q=1}^Q Fy_q \times Y_q + \sum_{k=1}^K Fw_k \times W_k + \sum_{e=1}^E Fr_e \times R_e + \sum_{s=1}^S Fv_s \times V_s \quad (22)$$

$$\begin{aligned} z_2 = & \sum_{i=1}^I \sum_{j=1}^J CI_i \times X_{ij} + \sum_{j=1}^J \sum_{k=1}^K Cu_{1j} \times Go_{jk} + \sum_{j=1}^J \sum_{e=1}^E Cu_{2j} \times Gr_{je} \\ & + \sum_{q=1}^Q \sum_{m=1}^M Cy_q \times D_{qm} + \sum_{k=1}^K \sum_{n_1=1}^{N_1} Cw_k \times P_{kn_1} \\ & + \sum_{e=1}^E Cr_e \left(\sum_{n_2=1}^{N_2} O_{en_2} + \sum_{s=1}^S Oc_{es} \right) + \sum_{s=1}^S \sum_{n_3=1}^{N_3} Cv_s \times L_{sn_3} \end{aligned} \quad (23)$$

$$\begin{aligned} z_3 = & \sum_{i=1}^I \sum_{j=1}^J CX_{ij} \times X_{ij} + \sum_{j=1}^J \sum_{k=1}^K CK_{jk} \times Go_{jk} + \sum_{j=1}^J \sum_{q=1}^Q CJ_{jq} \times Gw_{jq} \\ & + \sum_{e=1}^E \sum_{s=1}^S CS_{es} \times Oc_{es} + \sum_{e=1}^E \sum_{n_2=1}^{N_2} CN_{en_2} \times Oc_{en_2} + \sum_{e=1}^E \sum_{q=1}^Q CQ_{eq} \times Ow_{eq} \\ & + \sum_{s=1}^S \sum_{n_3=1}^{N_3} Cl_{sn_3} \times L_{sn_3} + \sum_{k=1}^K \sum_{n_1=1}^{N_1} Cp_{kn_1} \times P_{kn_1} + \sum_{q=1}^Q \sum_{m=1}^M Cd_{qm} \times D_{qm} \end{aligned} \quad (24)$$

- The variables R_j , Y_q , W_k , Z_e , and V_s are binary decision variables that indicate whether certain elements in the model are active. For example, $R_j = 1$ if facility j is open, and 0 otherwise. The other variables in the model are continuous and positive, representing real numbers greater than or equal to zero, used for modeling flows, quantities, and other measurable factors.
- The constraints (2)-(7) address the finite capacities and demands of each center within the supply chain. Specifically, constraint (2) limits the quantity of products shipped from producers to distribution centers. Constraints (3)-(7) then ensure that the quantities of products delivered to processing centers, compost centers, pistachio

factories, oil extraction centers, and cosmetic factories do not exceed the capacities of these centers.

- Products sent from any center must not exceed the orders received by that center. Therefore, constraint (8) stipulates that the quantity of products shipped from a processing center must be less than or equal to the quantity delivered to other centers. Constraints (9)-(12) specify the quantities of open-mouth pistachios, raw kernels, and waste produced through the dehulling process at each processing center. Similarly, constraints (13)-(17) regulate the flows within pistachio factories, oil extraction centers, cosmetic factories, and composting centers.
- Finally, to ensure that customer demand is met, constraints (18)-(21) guarantee that the products delivered to pistachio customers, pistachio oil customers, cosmetic customers, and compost markets meet or exceed their respective demands.

3 Heuristic approach

In this section, we present the methodology employed to address the optimization problem. The proposed approach integrates priority-based encoding and the Variable Neighborhood Search (VNS) algorithm, which together form the core of our solution strategy. We begin by detailing the solution representation, which utilizes priority-based encoding to efficiently manage and rank decision variables, allowing for effective exploration of the solution space. This encoding scheme is crucial in guiding the search process and maintaining feasibility while optimizing the objective function. Following this, we delve into the VNS algorithm, a robust metaheuristic known for its flexibility and capacity to escape local optima through systematic changes in neighborhood structures. The algorithm dynamically adjusts between multiple neighborhood structures, each designed to explore different aspects of the solution landscape, thereby enhancing the search process. We will also discuss the specific configurations and parameters used, as well as the design of neighborhood operators tailored to the problem characteristics. This comprehensive approach aims to deliver a high-quality solution by leveraging the strengths of priority-based encoding and the adaptability of VNS.

3.1 Solution representation

In this work, a priority-based encoding scheme is used to represent the solution space effectively, drawing on chromosome-based representations where each gene corresponds to a node (source or depot) in the transportation problem [1]. The transportation problem is modeled as a tree structure, with nodes representing sources or depots and edges representing transportation arcs. The primary objective is to construct a transportation tree that meets all depot demands while minimizing total transportation costs.

Each chromosome encodes the priority of nodes, guiding the construction of the transportation tree by establishing the most cost-effective connections first. The decoding process starts by identifying the node with the highest priority in the chromosome and connecting it to the corresponding node with the lowest transportation cost. For instance, if Depot 1 has the highest priority, it is linked to the source that offers the

lowest cost connection to Depot 1. The shipment amount is determined by the minimum of the source's capacity and the depot's demand. Once the shipment is fulfilled, the depot's priority is updated (or set to zero if its demand is satisfied), and the process continues with the next highest priority node. This sequential process of arc appending and priority updating repeats until all depots' demands are satisfied, resulting in a complete transportation tree.

The supply chain is determined using a step-by-step, backward approach, starting from consumer demand and moving upward toward producers. This method ensures resource allocation is optimized at each stage, preventing excess or unnecessary material flow. For example, after determining the quantity of open-mouth pistachios required by consumers, we work backward to calculate the factories' processing needs, ensuring their capacities are not overloaded. This then informs how much material the processing centers must supply, aligning each upstream stage with the final demand and optimizing resource use throughout the chain.

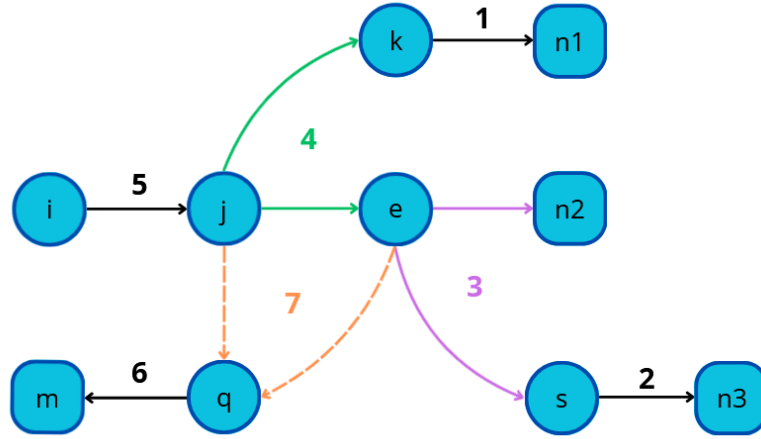


Fig. 1: Decodification steps of the transportation tree

As shown in Figure 1, the decoding process begins with the flow of pistachio products from factories to consumers, ensuring that all constraints are met: consumer demand is satisfied, the total amount of product leaving the factory does not exceed the pistachio received from processing centers multiplied by the factory's production rate, and the pistachio sent to any active factory is within its capacity. Likewise, the distribution of cosmetic products from factories to consumers and the transfer of oil from extraction centers to both oil consumers and cosmetic factories adhere to the same principles, ensuring all constraints are satisfied.

The flow from processing centers to pistachio factories and oil extraction centers is regulated by constraints ensuring that the total pistachios leaving a processing center do not exceed the harvested amount sent by producers. Harvested pistachios are divided into three parts, each assigned to a product type, with a $(1 - \beta)$ loss

factor applied to pistachios with shells and pits. Factories can receive more pistachios than they process, but must meet a minimum intake, while oil extraction centers send no more than $(1 - \lambda)$ of the kernels to consumers and cosmetics factories, with the remaining λ going to composting. All processing centers must respect their capacity limits to ensure efficient distribution.

The decoding process for this specific flow is more complex and requires a targeted approach. First, we must determine the flow of in-shell pistachios from processing centers to pistachio factories, and the flow of kernels from processing centers to oil extraction centers. Using the given constraints, we calculate the quantity of harvested pistachios each processing center must receive to meet the demands of both pistachio factories and oil extraction centers.

If the factory demand exceeds that of the oil extraction centers, the processing center must prioritize meeting the exact demand of the factories. Since harvested pistachios are divided into three proportional parts, basing the allocation solely on the oil extraction center's demand would leave the factories undersupplied. The priority should be given to the entity with the greatest need, adjusting the remaining flow accordingly, as constraints allow for over-supply but not shortages.

Thus, we need to adjust the quantity of kernels shipped from the processing centers to the oil extraction centers, ensuring enough is sent to satisfy the restrictions. This involves calculating the total amount of kernels required, identifying the oil extraction center with the lowest transportation cost, and directing the additional flow there to minimize costs.

If, instead, the oil extraction centers have higher demand, the process is reversed: the pistachio factories' flow is adjusted accordingly to balance the supply while minimizing transportation costs.

Next, the flow of harvested pistachios from producers to processing centers is also governed by constraints ensuring that no producer exceeds its production capacity and no processing center receives more than it can process. While the capacity of each producer limits the supply, the demand for each processing center has already been determined in a previous step. If producers' capacities are insufficient to meet the total demand, the solution becomes infeasible. However, due to the constraints on product allocation between processing centers, this flow will always satisfy the condition of equality.

Similarly, the flow of fertilizer from composting centers to consumers is governed by constraints that ensure composting centers do not receive more waste than they can handle. The amount of fertilizer sent must be less than or equal to the waste received from processing and oil extraction centers, adjusted by the composting center's production rate. Finally, for the waste flow from processing and oil extraction centers to composting centers, constraints also ensure that no composting center receives more waste than its capacity. Waste from each processing center is assigned to the composting center with the lowest associated cost, ensuring that no composting center receives more waste than required. If the waste sent to a composting center falls short of its needs, the demand is adjusted accordingly. Conversely, if excess waste is sent, the demand is set to zero. Once all processing centers have allocated their waste, any remaining composting needs are fulfilled by oil extraction centers, considering their

available capacity. If no additional waste is needed, no waste is sent from the oil extraction centers. Finally, the total cost of transporting waste is updated based on the assignments, ensuring efficient waste management across the supply chain.

3.2 VNS algorithm

The Variable Neighborhood Search (VNS) algorithm is a powerful metaheuristic that combines local search with systematic changes in neighborhood structures to explore the solution space effectively. By iteratively applying different neighborhood operators, VNS can escape local optima and discover high-quality solutions through continuous diversification and intensification phases.

The algorithm begins by initializing a solution and setting the maximum neighborhood size and time limit. It then enters a loop, where it repeatedly applies the shaking and local search procedures to generate new candidate solutions. If a better solution is found, it replaces the current best solution and resets the neighborhood size. Otherwise, the algorithm increases the neighborhood size and continues the search. The process repeats until the time limit is reached, at which point the best found solution is returned. Algorithm 1 provides a detailed outline of the VNS algorithm.

Algorithm 1 Variable Neighborhood Search (VNS)

Require: Initial solution x , maximum neighborhood size $k_{\max}$, time limit $t_{\max}$

Ensure: Best found solution x^*

```

1:  $x^* \leftarrow x$ 
2: repeat
3:    $k \leftarrow 1$ 
4:   repeat
5:      $x' \leftarrow \text{Shake}(x, k)$  ▷ Shaking
6:      $x'' \leftarrow \text{LocalSearch}(x')$  ▷ Local search
7:     if  $f(x'') < f(x^*)$  then
8:        $x^* \leftarrow x''$ 
9:        $k \leftarrow 1$  ▷ Reset neighborhood size
10:    else
11:       $k \leftarrow k + 1$ 
12:    end if
13:  until  $k > k_{\max}$ 
14:   $t \leftarrow \text{CpuTime}()$ 
15: until  $t > t_{\max}$ 

```

3.3 Neighborhood structures

In the context of the Variable Neighborhood Search (VNS) algorithm, neighborhood operators play a crucial role in exploring the solution space by systematically altering the current solution to generate new candidate solutions. The effectiveness of VNS relies heavily on the design and application of these operators, as they determine the

diversity and direction of the search process. In this study, we utilize four classic neighborhood operators: Swap, Reversion, Insertion, and Slide. Each operator perturbs the solution in a distinct manner, allowing the algorithm to escape local optima and explore different regions of the search space. By iteratively applying these operators, VNS enhances its capability to find high-quality solutions through continuous diversification and intensification phases. The neighborhood structures used are detailed below, and illustrated in Figure 2.:

- **Swap:** The swap operator exchanges the positions of two selected elements within the chromosome. In the example, the selected elements are 6 and 3. This operator is useful for making quick adjustments to the order of elements without altering the overall structure significantly.
- **Reversion:** The reversion operator reverses the order of all elements between the two selected indices. In this case, with elements 6 and 3 chosen, the operator reverses the sequence between these two points. This operation can effectively rearrange subsequences, potentially moving the solution closer to a local optimum by drastically altering a segment of the sequence.
- **Insertion:** In this operation, the unit in the second selected position is relocated immediately after the unit in the first position. Subsequently, all units that were originally between these positions are shifted accordingly to the right. This allows the algorithm to make focused adjustments by repositioning a specific element within the sequence while maintaining the overall order of the other elements.
- **Slide:** This operator begins by selecting two units of a solution randomly, like the other three. The first unit is removed, and the units that were initially located between the two selected units are shifted toward the start position of the first unit. Finally, the initially removed unit is placed at the position of the second unit. This sliding mechanism creates a subtle yet impactful reordering of the solution elements, which can enhance the search process by generating new configurations with minimal disruption.

4 Results

Our results demonstrate that the Variable Neighborhood Search (VNS) algorithm performs competitively against both an exact algorithm, Gurobi, and a population-based metaheuristic, the Genetic Algorithm (GA), when considering execution time and memory efficiency, particularly on larger instances. While the Exact Algorithm guarantees optimal solutions, its memory consumption and execution time increase significantly as problem sizes grow, often making it impractical for large-scale problems. VNS provides high-quality solutions with a significant reduction in both time and memory usage, making it a more practical choice in these scenarios. Although the population-based metaheuristic is widely used in the literature, its performance deteriorates with increasing problem size, both in terms of solution quality and execution time.

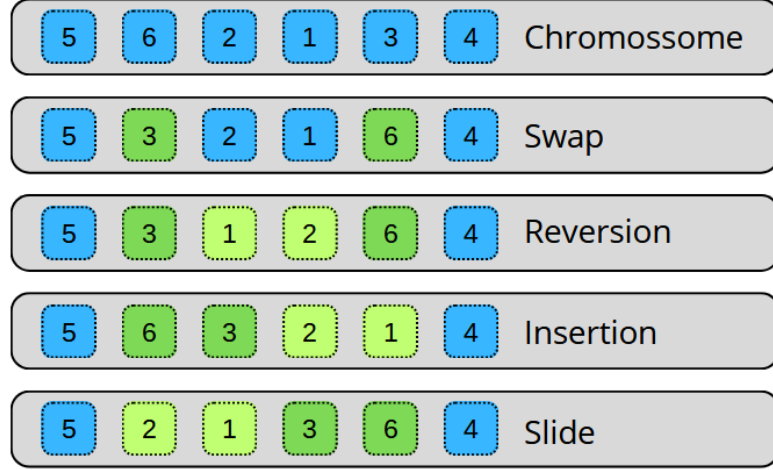


Fig. 2: Neighborhood structures used in the VNS Algorithm

4.1 Comparison between VNS and an Exact Algorithm

As shown in Table 1, Gurobi efficiently finds optimal solutions for smaller instances, such as 100 and 200, where the solution difference between VNS and Gurobi is relatively small—4.19% for 100 instances and 8.22% for 200 instances. However, as the problem size increases, the difference in solution quality becomes larger, with VNS being 20.12% away from the optimal solution at 800 instances.

The boxplots in Figure 2 further illustrate the variability of VNS across 30 runs compared to Gurobi’s single optimal solution. For instance 100, VNS results range from 7.15×10^6 to 8.75×10^6 , closely matching Gurobi’s solution of 7.06×10^6 with only a 4.19% difference. In instance 200, VNS shows slightly more variability, ranging from 1.40×10^8 to 1.75×10^8 , maintaining an 8.22% gap from Gurobi’s 1.37×10^8 . As problem size increases, the VNS range widens—for instance 400, from 2.70×10^8 to 3.30×10^8 , with a 13.62% difference, and for instance 800, from 5.40×10^8 to 6.85×10^8 , showing a 20.12% gap from Gurobi’s 5.47×10^8 . Although the difference in solution quality increases as the problem size grows, VNS remains competitive by providing solutions much faster than Gurobi, particularly for larger instances.

When examining execution time, the results clearly highlight Gurobi’s inefficiency on larger instances. For example, at 400 instances, Gurobi takes over 70,000 seconds to solve the problem, while VNS takes just over 2,000 seconds, offering a remarkable 97.13% reduction in time. By the time we reach 1600 instances, Gurobi is unable to find a solution due to its excessive resource demands, while VNS continues to provide a feasible solution in approximately 17.766 seconds.

These results emphasize that although the Exact Algorithm produces the best solutions for small instances, it becomes impractical for larger problems due to its memory usage and excessive execution times. VNS, while not optimal, offers a more practical and scalable solution for larger instances.

4.2 Comparison between VNS and GA

The performance of the Genetic Algorithm (GA) can be seen in Table 2. For small instances, such as 100, the difference between VNS and GA is negligible—only 0.71%. However, as the problem size grows, GA’s performance gradually worsens compared to VNS. By 800 instances, the difference in solution quality rises to 5.26%, indicating that VNS provides consistently better solutions for larger problems.

Statistical tests conducted on the results further validate that VNS outperforms GA in all tested instances. VNS not only exhibits a lower average solution value across small and large instances but also consistently produces more reliable and robust outcomes. The boxplots visually demonstrate VNS’s lower median values compared to GA, reinforcing the statistical tests’ conclusions. This shows that, although GA may perform competitively in smaller cases, such as in the 100-instance example, it falls significantly behind as the problem size increases, with VNS statistically surpassing GA in solution quality.

It is also important to highlight that GA takes considerably longer to run compared to VNS. The nature of GA, which relies on iterative population-based mechanisms such as selection, crossover, and mutation, demands a high number of epochs to reach an acceptable solution. This results in prolonged execution times, particularly for larger problem sizes. In contrast, VNS, with its focused exploration of neighborhood structures, efficiently narrows down the solution space, allowing for faster convergence. This significant difference in runtime makes VNS a more practical choice for larger problem instances, as GA’s prolonged execution time limits its scalability and applicability in real-world scenarios where timely solutions are crucial.

In summary, while GA can compete with VNS in smaller problem instances, it lags behind as the complexity increases. VNS consistently provides not only better solutions, as supported by statistical evidence, but also handles larger instances more efficiently, making it the more robust and scalable option for large-scale optimization problems.

Table 1: Comparison of the VNS algorithm and the Exact Algorithm (Gurobi) in terms of the average objective function value of the best solutions and the average execution time.

Instance	Objective function evaluation			Execution time		
	VNS	Exact Algorithm	Difference	VNS	Exact Algorithm	Difference
100	7.36×10^6	7.06×10^6	4.19%	241s	261s	-7.83%
200	1.49×10^8	1.38×10^8	8.22%	714s	3834s	-81.36%
400	3.08×10^8	2.71×10^8	13.62%	2016s	70339s	-97.13%
800	6.57×10^8	5.47×10^8	20.12%	5538s	11836s	-53.22%
1600	1.36×10^9	-	-	17767s	-	-

Table 2: Comparison of the VNS algorithm and a population-based metaheuristic (genetic algorithm) in terms of the average objective function value of the best solutions and the average execution time.

Instance	Objective function evaluation		
	VNS	GA	Difference
100	7.41×10^7	7.46×10^7	0.71%
200	1.50×10^8	1.55×10^8	3.37%
400	3.08×10^8	3.24×10^8	5.31%
800	6.57×10^8	6.92×10^8	5.26%
1600	1.36×10^9	-	-

5 Conclusion

The conclusion of this study underscores the advantages of the Variable Neighborhood Search (VNS) algorithm in optimizing the pistachio supply chain network. The proposed model effectively integrates various stages of the supply chain, from production to composting, ensuring that all capacity and demand constraints are satisfied. By minimizing total costs while considering transportation, production, and facility operations, the model addresses the complexities of managing pistachio products and by-products.

Our results demonstrate that VNS outperforms the Genetic Algorithm (GA) in both execution time and scalability, particularly as the problem size increases. While Gurobi provides optimal solutions for small instances, it becomes impractical for larger problems due to its higher memory consumption and longer processing times. In contrast, VNS offers a flexible, memory-efficient, and time-saving solution, delivering near-optimal results with significantly reduced computational demands.

Moving forward, research could benefit from the development of more specialized and problem-specific neighborhood operators for VNS, which may lead to even better performance by enabling more targeted exploration of the solution space. Additionally, combining VNS with exact algorithms for smaller problem instances presents an intriguing opportunity to balance accuracy and efficiency, creating a hybrid approach that could be more broadly applicable across industries. Statistical methods could also be integrated to further refine the search process, ensuring that both local and global optima are explored effectively.

By sharing the results and methodologies from this study, it is hoped that others in the field of supply chain optimization can build upon these insights. The adaptability of VNS, when combined with tailored heuristic techniques and exact methods, holds promise not only for the pistachio industry but for solving complex logistical challenges across various sectors. The potential for future research lies in expanding the versatility and precision of these algorithms, driving more efficient decision-making processes in increasingly intricate supply chains.

References

- [1] Gen, M., Altıparmak, F. & Lin, L. A genetic algorithm for two-stage transportation problem using priority-based encoding. *OR spectrum* **28**, 337–354 (2006).